


```

242         if start < 0:
243             logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
244             start = 0.0
245
246         if start >= end:
247             logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
248             skipped_segments.append((idx, start, end, "start >= end"))
249             continue
250
251         # Check if segment extends beyond video duration
252         if video_duration > 0:
253             if start >= video_duration:
254                 logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
255                             f"This segment cannot be extracted ? the video ran
out. Skipping.")
256                 skipped_segments.append((idx, start, end, "start beyond video
end"))
257                 continue
258
259             if end > video_duration:
260                 original_end = end
261                 end = video_duration
262                 logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
263                             f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")
264
265                 clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
266
267                 # Precise sub-second trim using fast re-encoding to preserve stream
headers
268                 padded_start = max(0.0, start - self.begin_padding)
269                 padded_end = end + self.end_padding
270
271                 # Clamp padded_end to video duration if known
272                 if video_duration > 0 and padded_end > video_duration:
273                     padded_end = video_duration
274                     logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
275                             f"Clamping to {video_duration:.2f}s (end padding
partially applied).")
276
277                 trim_duration = padded_end - padded_start
278                 logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
279
280                 base_cmd = [
281                     "ffmpeg", "-y",
282                     "-ss", str(round(padded_start, 3)),
283                     "-to", str(round(padded_end, 3)),
284                     "-i", raw_video_path,
285                     "-c:v", "libx264",
286                     "-preset", "superfast",
287                     "-crf", "22",
288                     "-c:a", "aac",
289                     "-b:a", "128k",
290                 ]
291                 vf_args = ["-vf", watermark_filter] if watermark_filter else []
292
293                 # Try with the watermark first; if that encode fails (e.g. a bad
font/logo
294                 # path), retry once WITHOUT it so a branding misconfig can never nuke

```

```

the
295         # whole reel. Clips stay codec-identical either way, so concat -c copy
holds.
296         attempts = [vf_args, []] if vf_args else [[]]
297         produced = False
298         last_exc: Optional[subprocess.CallProcessError] = None
299         cmd = base_cmd + [clip_path]
300         for attempt_vf in attempts:
301             cmd = base_cmd + attempt_vf + [clip_path]
302             try:
303                 subprocess.run(cmd, capture_output=True, check=True)
304             except subprocess.CallProcessError as e:
305                 last_exc = e
306                 if attempt_vf and vf_args:
307                     _err = (e.stderr.decode(errors="replace").strip()[-300:]
308                             if getattr(e, "stderr", None) else "")
309                     logger.warning(
310                         f"{segment_label}: watermark encode failed; retrying
without it. "
311                         f"ffmpeg: {_err or '(no stderr captured)'}"
312                     )
313                     continue
314                     break
315                 if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
316                     produced = True
317                     if not attempt_vf and vf_args:
318                         logger.warning(f"{segment_label}: encoded WITHOUT the
watermark (the watermark filter failed).")
319                         break
320                 last_exc = None # ffmpeg succeeded but produced nothing
321                 break
322
323         if produced:
324             temp_clips.append(clip_path)
325         elif last_exc is not None:
326             stderr_msg = last_exc.stderr.decode(errors='replace').strip()
327             logger.error(f"{segment_label}: FFmpeg trim failed.")
328             print(f"  Command: {' '.join(cmd)}")
329             print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
330             skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
331             continue
332         else:
333             logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
334             skipped_segments.append((idx, start, end, "empty output clip"))
335             continue
336
337         # Report summary of skipped segments
338         if skipped_segments:
339             logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
340             for seg_idx, seg_start, seg_end, reason in skipped_segments:
341                 logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
342             logger.info("")
343
344         if not temp_clips:
345             raise RuntimeError(
346                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
347                 f"All {len(segments)} segments were skipped. "
348                 f"Check the errors above for details. "
349                 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."

```

```

350         )
351
352         logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
353
354         # Create a list file for FFmpeg concat demuxer
355         list_file_path = os.path.join(tmp_dir, "clips.txt")
356         with open(list_file_path, "w") as f:
357             for clip in temp_clips:
358                 # Escape single quotes in path
359                 safe_path = clip.replace("'", "'\\'")

```

4. assistant (2026-06-18T07:52:23.944Z)

I've read the core logic. Now let me understand the broader context: how `watermark\_filter` gets set (to confirm the watermark path is reachable in production) and check the related test file.

5. user (2026-06-18T07:52:25.364Z)

```

180         f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181         f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
182     )
183
184     if logo_path and drawtext:
185         return
186     f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
187     if logo_path:
188         return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
189     return drawtext
190
191     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
192         """
193         Extracts selected segments from a raw video and stitches them together
194         into a seamless highlights file, applying end_padding to prevent abrupt endings.
195         """
196         if not segments:
197             raise ValueError("No highlight segments provided to compile.")
198
199         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
200         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
201         original_count = len(segments)
202         segments = self._merge_overlapping(segments)
203         if len(segments) < original_count:
204             logger.info(f"Merged {original_count - len(segments)} overlapping/duplicate segment(s) before stitching: %d
-> %d.",
205                 original_count - len(segments), original_count, len(segments))
206         if not segments:
207             raise ValueError("No valid (start < end) highlight segments after merge.")
208
209         if not os.path.exists(raw_video_path):
210             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
211
212         # Defense-in-depth: never let the output name escape output_dir (the API also
213         # validates this at the request boundary).
214         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
215
216         # Probe video duration so we can detect out-of-range segments
217         video_duration = self._get_video_duration(raw_video_path)
218         if video_duration > 0:
219             logger.info(f"Source video duration: {video_duration:.2f}s")
220         else:
221             logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")

```

```

221
222         # We will create temporary clips and stitch them using FFmpeg concat demuxer
223         temp_clips = []
224         skipped_segments = []
225
226         # Create a temp directory within output_dir
227         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
228             logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
229             # Build the (optional) watermark filtergraph once; it is applied during
every
230             # per-clip re-encode so the mark is baked into each clip identically.
231             watermark_filter = self._watermark_filter(tmp_dir)
232             if watermark_filter:
233                 logger.info("[watermark] applying branding overlay to each clip.")
234             for idx, (raw_start, raw_end) in enumerate(segments):
235                 # Normalize timestamps to float seconds ? handles float, int,
236                 # plain numeric strings, and colon-separated formats like "MM:SS"
237                 start = self._parse_time(raw_start)
238                 end = self._parse_time(raw_end)
239                 segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"

```

6. user (2026-06-18T07:52:25.730Z)

tests\test_compiler.py

7. assistant (2026-06-18T07:52:29.440Z)

Now let me trace the exact reachability logic. The candidate claims that on a clean exit (code 0) with empty output, the loop hits `last\_exc = None; break` after the first attempt, never trying `attempts[1]=[]`. Let me re-read the loop carefully and check the test file for any coverage of this scenario.

8. user (2026-06-18T07:52:31.336Z)

No matches found

9. assistant (2026-06-18T07:52:34.320Z)

The test file doesn't cover the watermark fallback or empty-output scenario at all. Now let me look at how `\_watermark\_filter` decides to return a non-empty filter, to confirm the watermark path (and thus `attempts = [vf\_args, []]`) is reachable in production.

10. user (2026-06-18T07:52:35.371Z)

```

120         return float(result.stdout.strip())
121     except Exception as e:
122         logger.warning(f"Could not determine video duration via ffprobe: {e}")
123         return 0.0
124
125     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
126         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
127         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
128         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
129         because the mark is already baked into every clip identically."""
130         wm = self.watermark
131         if not wm or not wm.get("enabled"):
132             return None
133
134         text = (wm.get("text") or "").strip()
135         logo_path = (wm.get("logo_path") or "").strip()
136         if logo_path and not os.path.exists(logo_path):
137             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo

```

```

overlay.")
138         logo_path = ""
139         if not text and not logo_path:
140             return None
141
142         margin = int(wm.get("margin", 24))
143         opacity = float(wm.get("opacity", 0.85))
144         position = str(wm.get("position", "bottom-right"))
145         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
146         dt_xy = {
147             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
148             "bottom-left": f"x={margin}:y=h-th-{margin}",
149             "top-right": f"x=w-tw-{margin}:y={margin}",
150             "top-left": f"x={margin}:y={margin}",
151         }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
152         ov_xy = {
153             "bottom-right": f"W-w-{margin}:H-h-{margin}",
154             "bottom-left": f"{margin}:H-h-{margin}",
155             "top-right": f"W-w-{margin}:{margin}",
156             "top-left": f"{margin}:{margin}",
157         }.get(position, f"W-w-{margin}:H-h-{margin}")
158
159         drawtext = ""
160         if text:
161             ratio = float(wm.get("font_size_ratio") or 0.035)
162             divisor = max(1, round(1.0 / ratio))
163             # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
164             tf_path = os.path.join(tmp_dir, "watermark.txt")
165             with open(tf_path, "w", encoding="utf-8") as fh:
166                 fh.write(text)
167             # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
168             # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
169             font_path = (wm.get("font_path") or "").strip()
170             if not font_path and _BUNDLED_FONT.exists():
171                 font_path = str(_BUNDLED_FONT)
172             if font_path:
173                 font_opt = f"fontfile='{self._ff_quote(font_path)}'"
174             else:
175                 font_opt = f"font='{wm.get('font', 'Sans')}'"
176             box = ""
177             if wm.get("box", True):
178                 box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
179             drawtext = (
180                 f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181                 f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
182             )
183
184             if logo_path and drawtext:
185                 return
186             f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
187             if logo_path:
188                 return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
189             return drawtext
190
191         def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
192             """
193             Extracts selected segments from a raw video and stitches them together
194             into a seamless highlights file, applying end_padding to prevent abrupt endings.
195             """

```

Now I have full understanding. Let me trace through the loop precisely for the candidate scenario.

```
**Scenario:** Watermark enabled with valid text/logo ? `watermark_filter` is non-empty ?  
`vf_args = ["-vf", watermark_filter]` ? `attempts = [vf_args, []]`.
```

```
**Iteration 1** (`attempt_vf = vf_args`):  
- `subprocess.run(cmd, ..., check=True)` ? exits 0 (no `CalledProcessError`), so the `except`  
block is skipped entirely.  
- Reaches line 315: `if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:` ?  
**False** (empty/missing output per the trigger).  
- Line 320: `last_exc = None`.  
- Line 321: `break`.
```

The loop breaks after the first attempt, never reaching `attempts[1] = []`. After the loop: `produced` is `False`, `last\_exc` is `None` ? falls to the `else` branch (line 332-335), logs "exited successfully but output clip is missing or empty," and `continue`s ? ****the segment is dropped****, never trying the un-watermarked fallback.

So the candidate's reachability analysis of the code path is ****correct****: a clean exit with empty output does break out without trying attempt 2.

Now the critical reachability question for a **real** defect: ****is the trigger condition realistic?*** The trigger is "a filtergraph ffmpeg accepts but that yields no output frames" while exiting 0. Let me assess this honestly.

For ffmpeg with `-vf`, if the filtergraph is malformed or references a missing/invalid resource (bad font, bad logo), ffmpeg almost always exits ****non-zero**** ? which the `CalledProcessError` path correctly handles (retries without watermark). The candidate's scenario requires ffmpeg to exit ****0**** yet produce a zero-byte/missing file.

Let me verify whether `subprocess.run` with `check=True` and a zero-frame-but-exit-0 case is genuinely achievable. The key question: the input clip always has frames (it's a trim of a real video over a validated `start < end` range). A watermark `-vf` overlay/drawtext on a non-empty input that ffmpeg **accepts** will produce output frames. The only way to get exit-0-but-empty is something like a write failure that ffmpeg doesn't flag, or a filter that drops all frames ? neither is a plausible watermark misconfiguration.

Let me confirm there isn't some other reachable trigger by checking the no-watermark path too. Note that the `last\_exc = None; break` at line 320-321 is the ****shared**** exit-on-empty path for **both** watermarked and un-watermarked single-attempt cases ? it's the intended "ffmpeg succeeded but produced nothing ? hard skip" handler. The candidate is essentially arguing this hard-skip fires one attempt too early when watermarking.

Let me double check the broader call path to see whether watermark is ever enabled by default / in production config.

12. user (2026-06-18T07:52:52.799Z)

```
10:# Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the overlay  
12:# `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives at  
18:         watermark: Optional[Any] = None):  
24:     self.watermark = self._normalize_watermark(watermark)  
28:     def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:  
29:         if watermark is None:  
31:             if hasattr(watermark, "model_dump"):  
32:                 return watermark.model_dump()  
33:             if isinstance(watermark, dict):  
34:                 return dict(watermark)  
47:         on-by-default bundled-font watermark (and absolute logo paths) on Windows.""  
125:     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:  
126:         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text  
128:         watermarking is disabled or has nothing to draw. The concat stage stays -c copy  
130:         wm = self.watermark  
137:         logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
```

```

overlay.")
164:         tf_path = os.path.join(tmp_dir, "watermark.txt")
229:         # Build the (optional) watermark filtergraph once; it is applied during every
231:         watermark_filter = self._watermark_filter(tmp_dir)
232:         if watermark_filter:

```

[Showing results with pagination = limit: 20]

13. assistant (2026-06-18T07:53:04.732Z)

I have enough to render a verdict on the REACHABILITY lens.

****Code-path reachability: CONFIRMED.**** The candidate's reading of the loop is accurate. When watermarking is enabled (``attempts = [vf_args, []]``), if attempt 1 (``subprocess.run`` with ``-vf``) exits 0 but the output file is missing/zero-byte, control reaches line 320-321 (``last_exc = None; break``) and breaks the loop ? ``attempts[1] = []`` (the un-watermarked fallback) is never tried. The segment is then hard-skipped at line 332-335. So the documented guarantee ("a branding misconfig can never nuke the whole reel") is not honored *for the specific case of exit-0-but-empty-output*. The watermark path is reachable: it's enabled whenever ``watermark.enabled`` is true with text or a valid logo (the comment at line 47 confirms it's an "on-by-default bundled-font watermark").

****However, the *trigger* condition is the weak link.**** The defect only manifests if ffmpeg exits 0 *and* yet produces an empty/missing clip from a watermark filtergraph. In practice, watermark misconfigurations (bad font, missing logo, malformed filter) cause ffmpeg to exit ****non-zero****, which is correctly handled by the ``CalledProcessError`` retry path (lines 304-313) ? that fallback works fine. A filtergraph that ffmpeg *accepts* (exit 0) over a non-empty validated input clip will produce output frames; getting exit-0-with-empty-output specifically from the ``-vf`` watermark step (vs. the un-watermarked step which faces the same input) is not a realistic watermark-misconfiguration outcome. An empty-output-on-exit-0 would near-certainly affect the un-watermarked attempt identically (disk full, etc.), so the fallback wouldn't help anyway.

So: the code path is genuinely reachable and the logic asymmetry is real, but the concrete input that distinguishes it from the already-handled error path is not plausibly reachable in production. This is a theoretical gap, not a reachable defect with a realistic trigger.

14. user (2026-06-18T07:53:14.968Z)

Structured output provided successfully

15. assistant (2026-06-18T07:53:22.088Z)

Verdict recorded. Summary for the REACHABILITY lens:

The candidate's reading of the loop logic at ``backend/pipeline/stitcher/compiler.py:300-321`` is accurate, and the watermark code path is genuinely reachable (watermark is described as on-by-default). The asymmetry is real: a clean ffmpeg exit (code 0) with empty/missing output does ``last_exc = None; break`` after attempt 1, bypassing ``attempts[1] = []``, so the documented "branding misconfig can never nuke the whole reel" guarantee is not honored for that specific case ? and the test file has zero coverage of it.

But the ****trigger is not reachably realistic****: watermark misconfigs (bad font/logo/filtergraph) make ffmpeg exit **non-zero**, which the ``CalledProcessError`` path already handles correctly. A ``-vf`` filtergraph that ffmpeg *accepts* (exit 0) over a non-empty validated input clip will produce frames; an exit-0-with-empty-output is an environmental failure (disk full, etc.) that would hit the un-watermarked attempt identically ? so the fallback wouldn't help anyway. No plausible watermark-specific input distinguishes this case from the already-handled error path.

``real=true, reachable=false, already_guarded_or_tested=false, intended=false``.